ANÁLISE LÉXICA EM PYTHON

Ciência da Computação

Disciplina: Compiladores

Professor: Adonias Caetano



INSTITUTO FEDERAL
Ceará

INSTITUTO FEDERAL
Ceará

OBJETIVOS

I. Introduzir formas de implementação de compiladores em Python;

II. Apresentar a biblioteca PLY como ferramenta de construção de analisadores léxicos;

III. Exemplificar um analisador léxico para de expressões aritméticas;



INSTITUTO FEDERAL
Ceará

CONTEÚDO

- Introdução
- Visão geral do PLY
- Código 01: Calclex.py
- Tokens pelo PLY
- Outros recursos do Lex
- Código 02: tokrules.py
- Código 03: MyLexer.py
- Manutenção do estado
- Clonagem Lexer
- Condições do Lexer
- Questões diversas

INTRODUÇÃO

- Nesta aula, usaremos a biblioteca **PLY** para realizar a implementação de analisadores léxicos em Python;

- Existe também a biblioteca **RPLY.**



```
mov     rax, 0x0a687f
push    rax
xor     rax, rax
mov     rsi, rsp
mov     rdi, 1
mov     rdx, 8
call    write
```

VISÃO GERAL DO PLY



INSTITUTO
FEDERAL
Ceará

INSTITUTO FEDERAL
Ceará

PLY

É uma implementação em Python puro das populares ferramentas de construção de compiladores **LEX** e **YACC**.

É bastante fiel à maneira como as ferramentas tradicionais **LEX** / **YACC** funcionam.

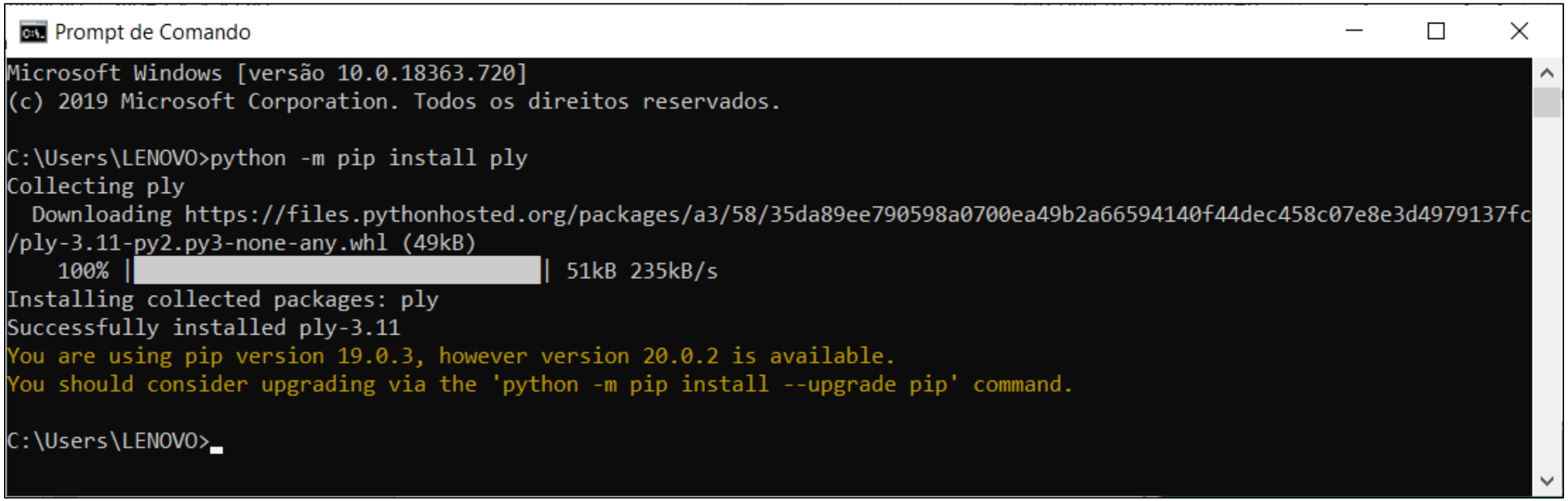
Isso inclui o suporte à análise **LALR(1)**, além de fornecer validação de entrada extensiva, relatório de erros e diagnóstico.

INSTALAÇÃO DO PLY

INSTITUTO FEDERAL
Ceará

- **Instalação no Windows:**

```
python –m pip install ply
```



VANTAGENS VS DESVANTAGENS

INSTITUTO FEDERAL
Ceará

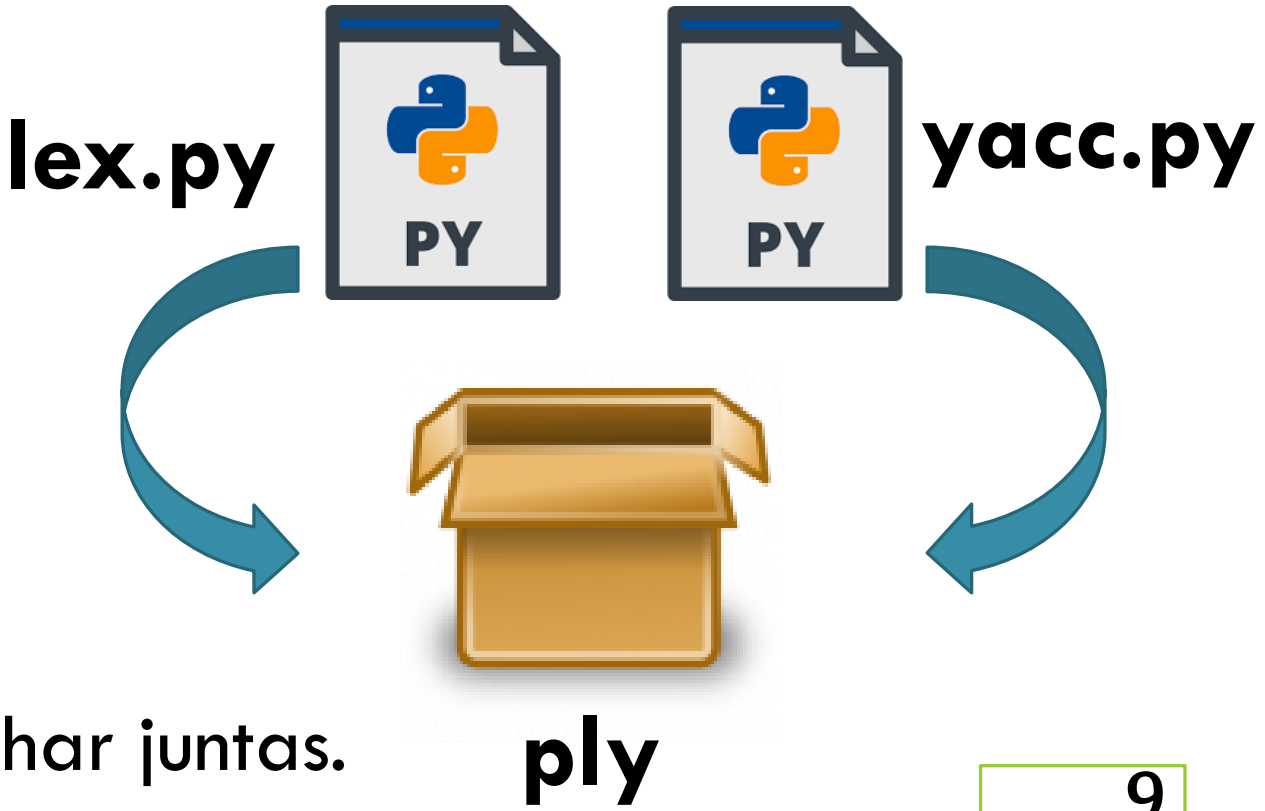
VANTAGENS	DESVANTAGENS
Por ser uma **ferramenta instrucional,** apresenta especificação de regras de token e gramática bastante exigente.	Não fornece muito em determinados recursos, como **construção automática de árvores de sintaxe abstrata e passagem de árvores.**
Detecta erros comuns de programação cometidos por usuários iniciantes.	Também não considera uma estrutura de análise.
Usuários avançados também acharão esses recursos úteis ao criar gramáticas complicadas para linguagens de programação reais.	É possível achar uma implementação **lex/yacc** simples e totalmente capaz, escrita inteiramente em Python.

O PACOTE PY

INSTITUTO FEDERAL
Ceará

- **PLY** consiste em dois módulos separados:

MÓDULO	FINALIDADE
lex.py	Divide o texto de entrada em uma coleção de tokens de acordo com regras de ER.
yacc.py	Reconhece a sintaxe da linguagem especificada na forma de uma gramática livre de contexto.



- As duas ferramentas devem trabalhar juntas.

MÓDULO LEX.PY

INSTITUTO FEDERAL
Ceará

- Especificamente, o **lex.py** fornece uma interface externa na forma de uma função **token** () que retorna o próximo token válido no fluxo de entrada.

```
1  # Tokenize
2  while True:
3      tok = lexer.token()
4      if not tok:
5          break      # No more input
6      print(tok)
```

AUSÊNCIA DE ARQUIVOS DE ESPECIFICAÇÃO

INSTITUTO FEDERAL
Ceará

Lex / Yacc Tradicionais	**PLY**
Requer um arquivo de entrada especial que é convertido em um arquivo de origem separado	As especificações fornecidas ao PLY são programas Python válidos.

Isso significa que não há arquivos de origem extras nem uma etapa especial de construção do compilador.

CÓDIGO 01: CALCLEX.PY



INSTITUTO
FEDERAL
Ceará

MÓDULO LEX

INSTITUTO FEDERAL
Ceará

- **lex.py** é usado para tokenizar uma sequência de entrada.
- **Exemplo de string de entrada:**

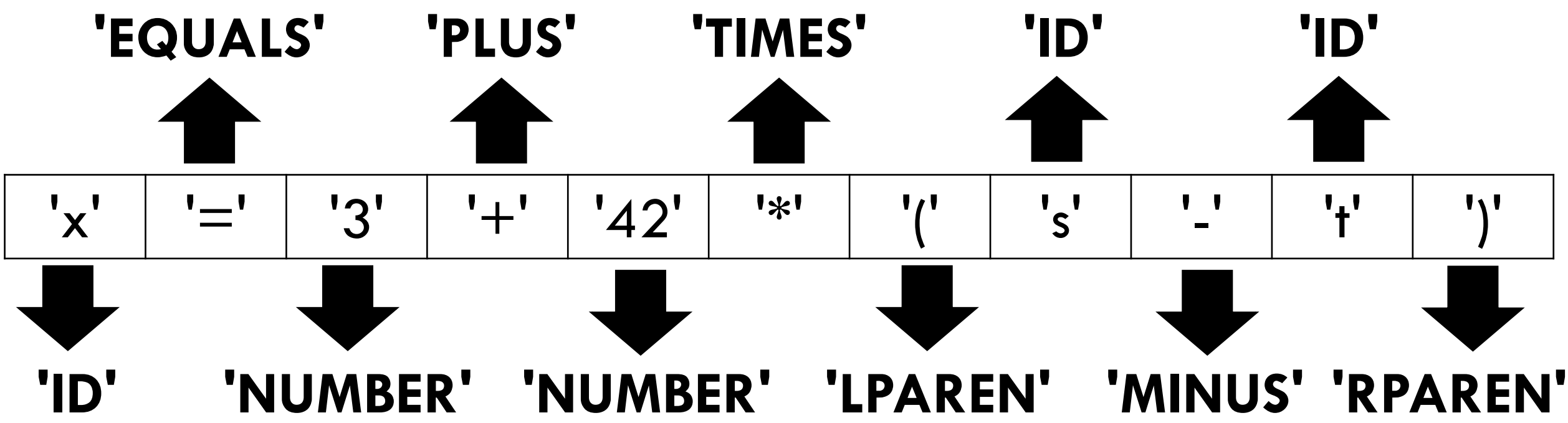
```
x = 3 + 42 * (s - t)
```

- Um tokenizador divide a string em tokens individuais

'x'	'='	'3'	'+'	'42'	'*'	'('	's'	'-'	't'	')'

MÓDULO LEX

- Os tokens geralmente recebem nomes para indicar o que são.



MÓDULO LEX

INSTITUTO FEDERAL
Ceará

- Mais especificamente, a entrada é dividida em pares de tipos e valores de token.
- Por exemplo:

('ID','x')	('EQUALS', '=')	('NUMBER','3')
('PLUS','+')	('NUMBER', '42')	('TIMES','*')
('LPAREN','(')	('ID','s')	('MINUS','-')
('ID','t')	('RPAREN',')')	

- A identificação de tokens geralmente é feita escrevendo uma série de regras de expressão regular.

INSTITUTO FEDERAL
Ceará

CALCLEX.PY

- Este exemplo mostra como o **lex.py** é usado para escrever um tokenizador simples.

```
# ------------------------------------------------------------
# calclex.py
#
# tokenizer for a simple expression evaluator for
# numbers and +,-,*,/
# ------------------------------------------------------------
import ply.lex as lex

# List of token names.   This is always required
tokens = (
   'NUMBER',
   'PLUS',
   'MINUS',
   'TIMES',
   'DIVIDE',
   'LPAREN',
   'RPAREN',
)

# Regular expression rules for simple tokens
t_PLUS    = r'\+'
t_MINUS   = r'-'
t_TIMES   = r'\*'
t_DIVIDE  = r'/'
t_LPAREN  = r'\('
t_RPAREN  = r'\)'
```

```
28  # A regular expression rule with some action code
29  def t_NUMBER(t):
30      r'\d+'
31      t.value = int(t.value)
32      return t
33
34  # Define a rule so we can track line numbers
35  def t_newline(t):
36      r'\n+'
37      t.lexer.lineno += len(t.value)
38
39  # A string containing ignored characters (spaces and tabs)
40  t_ignore  = ' \t'
41
42  # Error handling rule
43  def t_error(t):
44      print("Illegal character '%s'" % t.value[0])
45      t.lexer.skip(1)
46
47  # Build the lexer
48  lexer = lex.lex()
```

INSTITUTO FEDERAL
Ceará

CALCLEX.PY

TESTE DO LEXER

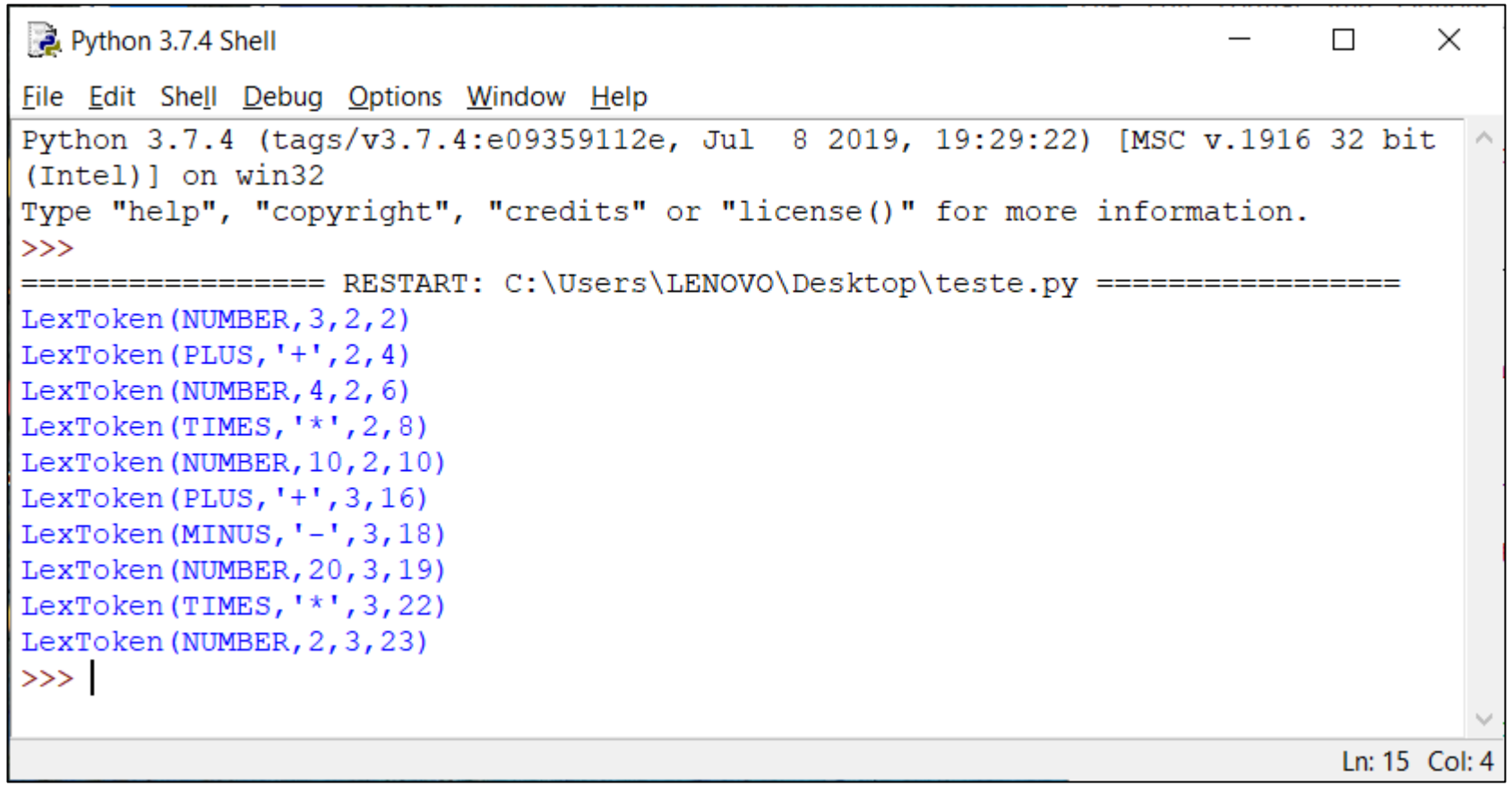
- Para usar o **lexer**, primeiro você precisa alimentar algum texto de entrada usando seu método **input** ().

- Depois disso, chamadas repetidas para **token** () produzem tokens.

```
50  # Test it out
51  data = '''
52  3 + 4 * 10
53    + -20 *2
54  '''
55
56  # Give the lexer some input
57  lexer.input(data)
58
59  # Tokenize
60  while True:
61      tok = lexer.token()
62      if not tok:
63          break      # No more input
64      print(tok)
65
66
```

EXECUÇÃO DO LEXER

- Quando executado, o exemplo produzirá a seguinte saída:



ANÁLISE DO CALCLEX.PY

INSTITUTO FEDERAL
Ceará

- É possível escrever o loop do **Tokenize** da seguinte maneira:

```
1  # Tokenize
2  while True:
3      tok = lexer.token()
4      if not tok:
5          break      # No more input
6      print(tok)
```



```
1  for tok in lexer:
2      print(tok)
```

ANÁLISE DO CALCLEX.PY

INSTITUTO FEDERAL
Ceará

- Os tokens retornados por **lexer.token** () são instâncias do LexToken.
- Este objeto possui atributos **tok.type, tok.value, tok.lineno** e **tok.lexpos.**

```
1  # Tokenize
2  while True:
3      tok = lexer.token()
4      if not tok:
5          break      # No more input
6      print(tok.type, tok.value, tok.lineno, tok.lexpos)
```

ATRIBUTOS DO TOKEN

INSTITUTO FEDERAL
Ceará

ATRIBUTO	FINALIDADE
tok.type	Contêm o tipo do Token É o tipo de token (como uma sequência). Por padrão, t.type é definido como o nome após o prefixo t_.
tok.value	Contêm o valor do próprio token É o lexeme (o texto real corresponde).
tok.lexpos	É a posição do token em relação ao início do texto de entrada.
tok.lineno	É o número da linha atual Contêm informações sobre a localização do **token** junto com atributo tok.lexpos.

TOKENS PELO PLY



INSTITUTO
FEDERAL
Ceará

A LISTA DE TOKENS



A LISTA DE TOKENS

INSTITUTO FEDERAL
Ceará

- No exemplo, o seguinte código especificou os nomes dos tokens:

```
 9  # List of token names.   This is always required
10  tokens = (
11      'NUMBER',
12      'PLUS',
13      'MINUS',
14      'TIMES',
15      'DIVIDE',
16      'LPAREN',
17      'RPAREN',
18  )
```

ESPECIFICAÇÃO DE TOKENS

INSTITUTO FEDERAL
Ceará

Use o módulo **re** do Python.

Cada token é especificado por uma regra de expressão regular.

Cada uma das regras é definida com um prefixo especial **t_**

Para tokens simples, a ER pode ser especificada como cadeias de caracteres como **t_PLUS = r'\+'**

O nome após t_ deve corresponder exatamente a um dos nomes fornecidos nos tokens.

Se algum tipo de ação precisar ser executado, uma regra de token poderá ser especificada como uma função.

ESPECIFICAÇÃO DE TOKENS

INSTITUTO FEDERAL
Ceará

- Por exemplo, essa regra corresponde a números e converte a string em um número inteiro Python.

```
1  def t_NUMBER(t):
2          r'\d+'
3          t.value = int(t.value)
4          return t
```

- **Nota:** cadeias brutas de Python são usadas, pois são a maneira mais conveniente de escrever cadeias de expressão regulares.

ESPECIFICAÇÃO DE TOKENS

INSTITUTO FEDERAL
Ceará

A função de ação pode modificar o conteúdo do objeto **LexToken**, conforme apropriado.

No entanto, quando estiver pronto, o token resultante deverá ser retornado.

Se nenhum valor for retornado pela função de ação, o token será simplesmente descartado e o próximo token será lido.

ORDENAÇÃO DE TOKENS

INSTITUTO FEDERAL
Ceará

- Ao criar a expressão regular principal, as regras são adicionadas na seguinte ordem:

Todos os tokens definidos por funções são adicionados na mesma ordem em que aparecem no arquivo lexer.

Os tokens definidos por sequências de caracteres são ordenandos em ordem decrescente do comprimento da expressão regular (expressões mais longas são adicionadas primeiro).

ORDENAÇÃO DE TOKENS

INSTITUTO FEDERAL
Ceará

- Por exemplo, se você quiser ter tokens separados para "=" e "==", precisará verificar se "==" está marcado primeiro.

- A ordenação decrescente de comprimento resolve esse problema.

- Para funções, a ordem pode ser explicitamente controlada, pois as regras que aparecem primeiro são verificadas primeiro.

DEFININDO PALAVRAS RESERVADAS

INSTITUTO FEDERAL
Ceará

- Basta escrever uma única regra para corresponder a um identificador e fazer uma busca de nome especial em uma função como esta:

```
reserved = {
    'if' : 'IF',
    'then' : 'THEN',
    'else' : 'ELSE',
    'while' : 'WHILE',
    ...
}

tokens = ['LPAREN','RPAREN',...,'ID'] + list(reserved.values())

def t_ID(t):
    r'[a-zA-Z_][a-zA-Z_0-9]*'
    t.type = reserved.get(t.value,'ID')    # Check for reserved words
    return t
```

Essa abordagem reduz bastante o número de regras de expressão regular e provavelmente torna as coisas um pouco mais rápidas.

DEFININDO PALAVRAS RESERVADAS

INSTITUTO FEDERAL
Ceará

- **Nota:** Evite escrever regras individuais para palavras reservadas.
- Por exemplo, se você escrever regras como esta:

```
1 t_FOR   = r'for'
2 t_PRINT = r'print'
```



- Essas regras serão acionadas para identificadores que incluam essas palavras como prefixo, como “**forget**" ou “**printed**".
- Provavelmente não é isso que você deseja.

VALORES DE TOKENS

INSTITUTO FEDERAL
Ceará

- Normalmente, o atributo **value** é o texto correspondente, mas o valor pode ser atribuído a qualquer objeto Python.
- Por exemplo, pode-se retornar o nome e as informações do identificador de algum tipo de tabela de símbolos.

```
1  def t_ID(t):
2      ...
3      # Look up symbol table information and return a tuple
4      t.value = (t.value, symbol_lookup(t.value))
5      ...
6      return t
```

VALORES DE TOKENS

INSTITUTO FEDERAL
Ceará

```
1  def t_ID(t):
2      ...
3      # Look up symbol table information and return a tuple
4      t.value = (t.value, symbol_lookup(t.value))
5      ...
6      return t
```

- Portanto, acessar outros atributos pode ser desnecessariamente estranho.
- Se você precisar armazenar vários valores em um token, atribua uma tupla, dicionário ou instância ao valor.

TOKENS IGNORADOS

- Os espaços em branco sem escape são ignorados e os comentários são permitidos nesse modo.
- Se o seu padrão envolver espaço em branco, use **\s**.
- Se você precisar corresponder ao caractere #, use [#].



TOKENS IGNORADOS

INSTITUTO FEDERAL
Ceará

- Para descartar explicitamente um token, como um comentário, basta definir uma regra de token que não retorne valor. Por exemplo:

```
1  def t_COMMENT(t):
2      r'\#.*'
3      pass
4      # No return value. Token discarded
```

- Como alternativa, o prefixo "**ignore_**" na declaração do token força um token a ser ignorado. Por exemplo:

```
t_ignore_COMMENT = r'\#.*'
```

OUTROS RECURSOS DO LEX

INSTITUTO FEDERAL
Ceará

ATRIBUTOS DO LEXER

INSTITUTO FEDERAL
Ceará

- Um Lexer de objeto Lexer possui vários atributos internos que podem ser úteis em determinadas situações.

ATRIBUTO	FINALIDADE
`lexer.lexpos`	É um número inteiro que contém a posição atual no texto de entrada.
`lexer.lineno`	O valor atual do atributo de número da linha armazenado no lexer. PLY especifica apenas que o atributo existe - ele nunca define, atualiza ou executa qualquer processamento com ele.

ATRIBUTOS DO LEXER

INSTITUTO FEDERAL
Ceará

ATRIBUTO	FINALIDADE
`lexer.lexdata`	O texto de entrada atual armazenado no lexer. Esta é a string passada com o método input ().
`lexer.lexmatch`	Este é o objeto Match bruto retornado pela função **re.match** () do Python (usada internamente pelo PLY) para o token atual. **Nota:** Este atributo é atualizado apenas quando os tokens são definidos e processados por funções.

NÚMEROS DE LINHA E INFORMAÇÕES POSICIONAIS

INSTITUTO FEDERAL
Ceará

- Por padrão, o **lex.py** não sabe nada sobre números de linha, pois não sabe nada sobre o que constitui uma "linha" de entrada;
- **Exemplo:** o caractere de nova linha ou mesmo se a entrada é um texto.
- Para atualizar essas informações, você precisa escrever uma regra especial.
- No exemplo, a regra **t_newline** () mostra como fazer isso.

```
1  # Define a rule so we can track line numbers
2  def t_newline(t):
3      r'\n+'
4      t.lexer.lineno += len(t.value)
```

O atributo **lineno** do lexer **t.lexer** subjacente é atualizado.

NÚMEROS DE LINHA E INFORMAÇÕES POSICIONAIS

INSTITUTO FEDERAL
Ceará

- É possível calcular as informações da coluna como uma etapa separada.
- Por exemplo, basta contar para trás até chegar a uma nova linha.

```
1  # Compute column.
2  #     input is the input text string
3  #     token is a token instance
4  def find_column(input, token):
5      line_start = input.rfind('\n', 0, token.lexpos) + 1
6      return (token.lexpos - line_start) + 1
```

CARACTERES LITERAIS

INSTITUTO FEDERAL
Ceará

- Caracteres literais podem ser especificados definindo-se literais de variável no seu módulo de lexing. Por exemplo:

```
literals = [ '+','-','*','/' ]
```

- Ou alternativamente

```
literals = "+-*/"
```

- Um caractere literal é simplesmente um único caractere retornado "como está" quando encontrado pelo lexer.

CARACTERES LITERAIS

- Os literais são verificados após todas as regras de expressão regular definidas através de funções para cada literal.

```
1  literals = [ '{', '}' ]
2
3  def t_lbrace(t):
4      r'\{'
5      t.type = '{'      # Set token type to the expected literal
6      return t
7
8  def t_rbrace(t):
9      r'\}'
10     t.type = '}'      # Set token type to the expected literal
11     return t
```

Se uma regra começar com um dos caracteres literais, ela sempre terá precedência.

Quando um token literal é retornado, os atributos de tipo e valor são definidos para o próprio caractere.

TRATAMENTO DE ERROS

INSTITUTO FEDERAL
Ceará

- A função **t_error** () manipula erros de lexing quando caracteres ilegais são detectados.
- O atributo **t.value** contém o restante da sequência de entrada que não foi tokenizada.

```
1  # Error handling rule
2  def t_error(t):
3      print("Illegal character '%s'" % t.value[0])
4      t.lexer.skip(1)
```

- Aqui é impresso o caractere incorreto e avançamos um caractere chamando **t.lexer.skip (1)**.

TRATAMENTO DE EOF

INSTITUTO FEDERAL
Ceará

- A função **t_eof ()** é usada para manipular uma condição de fim de arquivo (EOF) na entrada.

```
1  # EOF handling rule
2  def t_eof(t):
3      # Get more input (Example)
4      more = raw_input('... ')
5      if more:
6          self.lexer.input(more)
7          return self.lexer.token()
8      return None
```

Como entrada, ele recebe um tipo de token **'eof'** com os atributos **lineno** e **lexpos** configurados adequadamente.

Retorna o próximo token disponível (chamando **self.lexer.token ()**) ou **None** para não indicar mais dados.

- O principal uso dessa função é fornecer mais informações ao lexer, para que ele possa continuar analisando.

MANUSEIO DE EOF

INSTITUTO FEDERAL
Ceará

- **Se você o estiver usando no relatório de erros, lembre-se de que:**

1. Definir mais entradas com o método **self.lexer.input()** NÃO redefine o estado **lexer** ou o atributo **lineno** usado para o rastreamento de posição.
2. O atributo **lexpos** é redefinido.



CONSTRUINDO E USANDO O LEXER

INSTITUTO FEDERAL
Ceará

- Para construir o lexer, a função **lex.lex ()** é usada.
- Por exemplo:

```
lexer = lex.lex()
```

- Essa função usa reflexão (ou introspecção) do Python para ler as regras de expressão regular fora do contexto de chamada e criar o lexer.

CONSTRUINDO E USANDO O LEXER

INSTITUTO FEDERAL
Ceará

- Dois métodos podem ser usados para controlar o lexer.

MÉTODO	FINALIDADE
lexer.input (dados)	Redefina o lexer e armazene uma nova string de entrada.
lexer.token ()	Retorne o próximo token. Retorna uma instância especial do LexToken em caso de sucesso ou Nenhum se o final do texto de entrada tiver sido atingido.

O DECORADOR @TOKEN

INSTITUTO FEDERAL
Ceará

- Em algumas aplicações, convém definir tokens de construção como uma série de regras de expressão regular mais complexas.

```
1  digit            = r'([0-9])'
2  nondigit         = r'([_A-Za-z])'
3  identifier       = r'(' + nondigit + r'(' + digit + r'|' + nondigit + r')*)'
4
5  def t_ID(t):
6      # want docstring to be identifier above. ?????
7      ...
```

- Deseja-se que a regra para o ID seja uma das variáveis acima.
- No entanto, não há como especificar isso diretamente usando uma sequência de documentação normal.

O DECORADOR @TOKEN

INSTITUTO FEDERAL
Ceará

- Para resolver esse problema, pode-se usar o decorador **@TOKEN**.

```
1  from ply.lex import TOKEN
2
3  @TOKEN(identifier)
4  def t_ID(t):
5      ...
```

- Isso anexará o identificador à documentação para **t_ID ()**, permitindo que o **lex.py** funcione normalmente.

MODO OTIMIZADO

- Para melhorar o desempenho, pode-se usar o modo otimizado do Python: executando o Python com a opção **-O**.

- No entanto, isso faz com que o Python ignore as strings da documentação (problemas para o **lex.py**).



MODO OTIMIZADO

INSTITUTO FEDERAL
Ceará

I. A solução é criar seu lexer usando a opção otimizar da seguinte maneira:

```
lexer = lex.lex(optimize=1)
```

II. Execute o Python em seu modo operacional normal.

Resultado: O **lex.py** gravará um arquivo chamado **lextab.py** no mesmo diretório do módulo que contém a especificação do lexer.

Este arquivo contém todas as regras e tabelas de expressão regular usadas durante a **lexing**.

O **lextab.py** será simplesmente importado para criar o lexer.

MODO OTIMIZADO

INSTITUTO FEDERAL
Ceará

- Essa abordagem melhora substancialmente o tempo de inicialização do lexer e funciona no modo otimizado do Python.
- Para alterar o nome do módulo gerado pelo lexer, use o argumento da palavra-chave **lextab**. Por exemplo:

```
lexer = lex.lex(optimize=1,lextab="footab")
```

- Ao executar no modo otimizado, é importante observar que o lex desativa a maioria das verificações de erros.

DEPURAÇÃO

INSTITUTO FEDERAL
Ceará

- Basta executar **lex** () no modo de depuração da seguinte maneira:

```
lexer = lex.lex(debug=1)
```

- Produz vários tipos de informações de depuração, incluindo todas as regras adicionadas, as expressões regulares principais usadas pelo lexer e os tokens gerados durante a lexing.

CÓDIGO 02: TOKRULES.PY



INSTITUTO
FEDERAL
Ceará

```
# module: tokrules.py
# This module just contains the lexing rules

# List of token names.   This is always required
tokens = (
    'NUMBER',
    'PLUS',
    'MINUS',
    'TIMES',
    'DIVIDE',
    'LPAREN',
    'RPAREN',
)

# Regular expression rules for simple tokens
t_PLUS    = r'\+'
t_MINUS   = r'-'
t_TIMES   = r'\*'
t_DIVIDE  = r'/'
t_LPAREN  = r'\('
t_RPAREN  = r'\)'
```

INSTITUTO FEDERAL
Ceará

```
23 # A regular expression rule with some action code
24 def t_NUMBER(t):
25     r'\d+'
26     t.value = int(t.value)
27     return t
28
29 # Define a rule so we can track line numbers
30 def t_newline(t):
31     r'\n+'
32     t.lexer.lineno += len(t.value)
33
34 # A string containing ignored characters (spaces and tabs)
35 t_ignore  = ' \t'
36
37 # Error handling rule
38 def t_error(t):
39     print("Illegal character '%s'" % t.value[0])
40     t.lexer.skip(1)
```

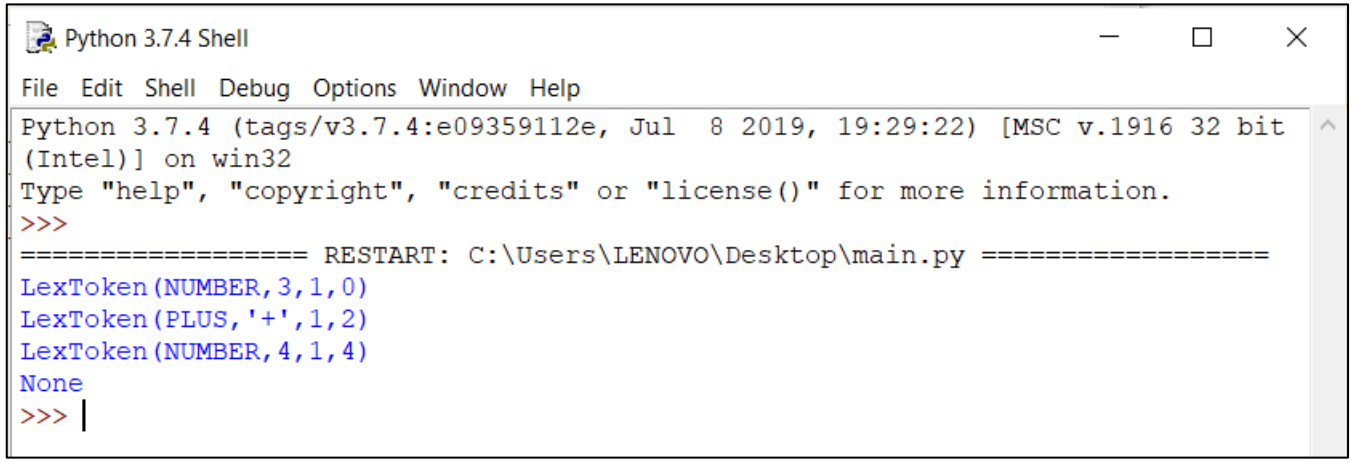
INSTITUTO FEDERAL
Ceará

REGRAS EM MÓDULOS DIFERENTES

INSTITUTO FEDERAL
Ceará

- Os lexers são especificados todos dentro de um módulo Python.
- O argumento **module** permite colocar regras de token em um módulo diferente daquele em que invoca lex ().

```
1  import tokrules
2  import ply.lex as lex
3
4  lexer = lex.lex(module=tokrules)
5
6  lexer.input("3 + 4")
7
8  print(lexer.token())
9  print(lexer.token())
10 print(lexer.token())
11 print(lexer.token())
```



CÓDIGO 03: MyLexer.PY



INSTITUTO
FEDERAL
Ceará

```
1  import ply.lex as lex
2
3  class MyLexer(object):
4      # List of token names.   This is always required
5      tokens = (
6         'NUMBER',
7         'PLUS',
8         'MINUS',
9         'TIMES',
10        'DIVIDE',
11        'LPAREN',
12        'RPAREN',
13     )
14
15     # Regular expression rules for simple tokens
16     t_PLUS    = r'\+'
17     t_MINUS   = r'-'
18     t_TIMES   = r'\*'
19     t_DIVIDE  = r'/'
20     t_LPAREN  = r'\('
21     t_RPAREN  = r'\)'
```

INSTITUTO FEDERAL
Ceará

- A opção do módulo também pode ser usada para definir lexers de instâncias de uma classe.

```
23  # A regular expression rule with some action code
24  # Note addition of self parameter since we're in a class
25  def t_NUMBER(self,t):
26      r'\d+'
27      t.value = int(t.value)
28      return t
29
30  # Define a rule so we can track line numbers
31  def t_newline(self,t):
32      r'\n+'
33      t.lexer.lineno += len(t.value)
34
35  # A string containing ignored characters (spaces and tabs)
36  t_ignore  = ' \t'
37
38  # Error handling rule
39  def t_error(self,t):
40      print("Illegal character '%s'" % t.value[0])
41      t.lexer.skip(1)
```

INSTITUTO FEDERAL
Ceará

```
43      # Build the lexer
44      def build(self,**kwargs):
45          self.lexer = lex.lex(module=self, **kwargs)
46
47      # Test it output
48      def test(self,data):
49          self.lexer.input(data)
50          while True:
51                tok = self.lexer.token()
52                if not tok:
53                    break
54                print(tok)
55
56  # Build the lexer and try it out
57  m = MyLexer()
58  m.build()           # Build the lexer
59  m.test("3 + 4")     # Test it
```

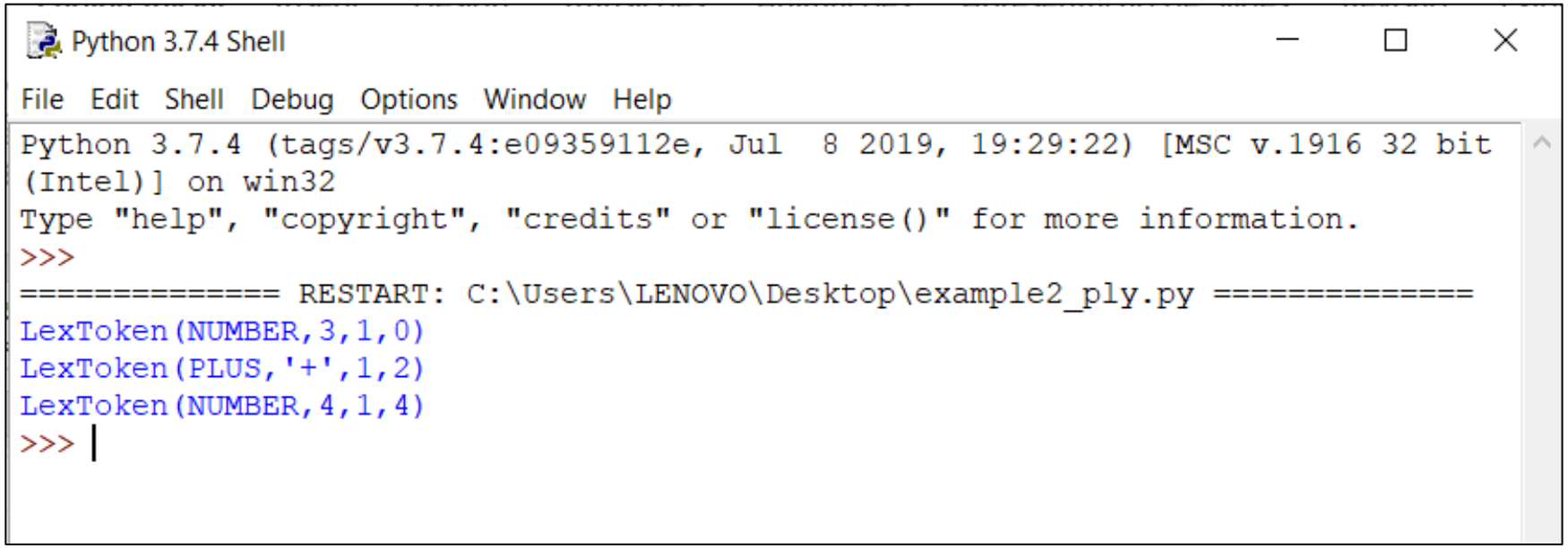
INSTITUTO FEDERAL
Ceará

Ao criar um lexer a partir da classe, você deve construí-lo a partir de uma instância da classe, não o objeto da classe em si.

Isso ocorre porque PLY só funciona corretamente se as ações do lexer são definidas por métodos vinculados.

REGRAS EM MÓDULOS DIFERENTES

INSTITUTO FEDERAL
Ceará



ESPECIFICAÇÃO ALTERNATIVA DE LEXERS

INSTITUTO FEDERAL
Ceará

- Ao usar a opção de módulo para **lex ()**, PLY coleta símbolos do objeto subjacente usando a função **dir ()**.

- Não há acesso direto ao atributo **__dict__** do objeto fornecido como um valor do módulo.

- Finalmente, se deseja manter as coisas bem encapsuladas, mas não deseja usar uma definição de classe completa, os lexers podem ser definidos usando **closures**.

```
1  import ply.lex as lex
2
3  # List of token names.   This is always required
4  tokens = (
5     'NUMBER',
6     'PLUS',
7     'MINUS',
8     'TIMES',
9     'DIVIDE',
10    'LPAREN',
11    'RPAREN',
12 )
13
14 def MyLexer():
15     # Regular expression rules for simple tokens
16     t_PLUS    = r'\+'
17     t_MINUS   = r'-'
18     t_TIMES   = r'\*'
19     t_DIVIDE  = r'/'
20     t_LPAREN  = r'\('
21     t_RPAREN  = r'\)'
```

INSTITUTO FEDERAL
Ceará

ESPECIFICAÇÃO ALTERNATIVA DE LEXERS

INSTITUTO FEDERAL
Ceará

```
23      # A regular expression rule with some action code
24      def t_NUMBER(t):
25          r'\d+'
26          t.value = int(t.value)
27          return t
28
29      # Define a rule so we can track line numbers
30      def t_newline(t):
31          r'\n+'
32          t.lexer.lineno += len(t.value)
33
34      # A string containing ignored characters (spaces and tabs)
35      t_ignore  = ' \t'
```

ESPECIFICAÇÃO ALTERNATIVA DE LEXERS

INSTITUTO FEDERAL
Ceará

```
37      # Error handling rule
38      def t_error(t):
39          print("Illegal character '%s'" % t.value[0])
40          t.lexer.skip(1)
41
42      # Build the lexer from my environment and return it
43      return lex.lex()
```

Nota importante:

- Se você estiver definindo um lexer usando uma classe ou fechamento, lembre-se de que o PLY ainda exige que você defina apenas um único lexer por módulo (arquivo de origem).
- Existem extensas partes de validação / verificação de erros do PLY que podem relatar falsamente mensagens de erro se você não seguir esta regra.

MANUTENÇÃO DO ESTADO

INSTITUTO FEDERAL
Ceará

USO DE VARIÁVEIS GLOBAIS

INSTITUTO FEDERAL
Ceará

Manter uma variedade de informações de estado, por exemplo, configurações de modo, tabelas de símbolos e outros detalhes.

- **Exemplo:** acompanhar quantos NUMBER tokens foram encontrados.

```
1  num_count = 0
2  def t_NUMBER(t):
3      r'\d+'
4      global num_count
5      num_count += 1
6      t.value = int(t.value)
7      return t
```

Uma solução é manter um conjunto de variáveis globais no módulo em que você criou o lexer.

USO DE ATRIBUTO LEXER

INSTITUTO FEDERAL
Ceará

- Uma outra solução para armazenar informações está dentro do objeto Lexer criado por **lex ()**.

Basta usar o atributo lexer de tokens passados para as várias regras.

```
1  def t_NUMBER(t):
2      r'\d+'
3      t.lexer.num_count += 1      # Note use of lexer attribute
4      t.value = int(t.value)
5      return t
6
7  lexer = lex.lex()
8  lexer.num_count = 0             # Set the initial count
```

USO DE ATRIBUTO LEXER

INSTITUTO FEDERAL
Ceará

É simples e funciona corretamente em aplicativos em que existem várias instanciações de um determinado lexer no mesmo aplicativo.

No entanto, isso também pode parecer uma violação grosseira do encapsulamento aos puristas de OO.

Basta armazenar atributos no lexer que não tenha nome:

- Começando com esse prefixo
- Que entre em conflito com um dos métodos predefinidos;

❑ **Por exemplo:** input (), token () entre outras.

USO DE CLASSES

- Uma terceira solução é defini-lo como uma classe:

```
1  class MyLexer:
2      ...
3      def t_NUMBER(self,t):
4          r'\d+'
5          self.num_count += 1
6          t.value = int(t.value)
7          return t
8
9      def build(self, **kwargs):
10         self.lexer = lex.lex(object=self,**kwargs)
11
12     def __init__(self):
13         self.num_count = 0
```

- ✓ É mais fácil de gerenciar se o seu aplicativo criar várias instâncias do mesmo lexer
- ✓ Se precisar gerenciar muitos estados.
- ✓ O estado também pode ser gerenciado através de fechamentos.

USO DE CLASSES

- Por exemplo, no Python 3:

```
1  def MyLexer():
2      num_count = 0
3      ...
4      def t_NUMBER(t):
5          r'\d+'
6          nonlocal num_count
7          num_count += 1
8          t.value = int(t.value)
9          return t
10     ...
```

CLONAGEM LEXER



INSTITUTO
FEDERAL
Ceará

CLONAGEM LEXER

INSTITUTO FEDERAL
Ceará

- Se necessário, um objeto lexer pode ser duplicado invocando seu método **clone** (). Por exemplo:

```
1 lexer = lex.lex()
2   ...
3 newlexer = lexer.clone()
```

- A cópia é exatamente idêntica ao lexer original, incluindo qualquer texto de entrada e estado interno.
- O clone permite que um conjunto diferente de texto de entrada seja fornecido, que pode ser processado separadamente.

CLONAGEM LEXER

INSTITUTO FEDERAL
Ceará

Pode ser útil em situações que envolve processamento recursivo ou reentrante.

Se precisar fazer uma varredura antecipada na entrada, pode-se criar um clone e usá-lo para olhar adiante.

Se for o caso de pré-processador, lexers clonados poderiam ser usados para manipular arquivos de entrada diferentes.

CLONAGEM LEXER

INSTITUTO FEDERAL
Ceará

- Os lexers recém-criados compartilharão todo esse estado com o lexer original.
- Por exemplo, se você definiu um lexer como uma classe e fez isso:

```
1  m = MyLexer()
2  a = lex.lex(object=m)      # Create a lexer
3
4  b = a.clone()              # Clone the lexer
```

- Então, **a** e **b** serão vinculados ao mesmo objeto **m**, e quaisquer alterações em **m** serão refletidas nos dois lexers.

CLONAGEM LEXER

INSTITUTO FEDERAL
Ceará

clone()

Cria um novo lexer que reutiliza as expressões regulares e o ambiente de outro lexer.

lex.lex()

Faz uma cópia totalmente nova de um lexer.

CONDIÇÕES DO LEXER



INSTITUTO
FEDERAL
Ceará

CONDIÇÕES DE LEXING E INÍCIO CONDICIONAIS

INSTITUTO FEDERAL
Ceará

- Em algumas aplicações, pode ser útil ter diferentes estados de lexing.

Exemplo de aplicação: a ocorrência de um determinado token ou construção sintática desencadeie um tipo diferente de lexing.

Por exemplo: quando estiver lendo uma string o \t ou \n não serão ignorados.

- O PLY suporta um recurso que permite que o lexer subjacente seja colocado em uma série de estados diferentes.

CONDIÇÕES DE LEXING E INÍCIO CONDICIONAIS

INSTITUTO FEDERAL
Ceará

- Cada estado pode ter seus próprios tokens, regras de lexing e assim por diante.
- A implementação é baseada principalmente no recurso "condição inicial" do GNU flex.
- Detalhes sobre isso podem ser encontrados em:

http://flex.sourceforge.net/manual/Start-Conditions.html.

CONDIÇÕES DE LEXING E INÍCIO CONDICIONAIS

INSTITUTO FEDERAL
Ceará

- Para definir um novo estado de lexing basta incluir uma declaração de "estados" no seu arquivo lex. Por exemplo:

```
1 states = (
2       ('foo','exclusive'),
3       ('bar','inclusive'),
4  )
```

- Esta declaração declara dois estados, **'foo'** e **'bar'**.
- Estados podem ser de dois tipos; **'exclusive'** e **'inclusive'**.

TIPOS DE ESTADOS	FINALIDADE
exclusive	Substitui completamente o comportamento padrão do lexer, ou seja, lex retornará apenas tokens e aplicará regras definidas especificamente para esse estado
inclusive	Adiciona tokens e regras adicionais ao conjunto de regras padrão. lex retornará os tokens definidos por padrão, além daqueles definidos para o estado inclusivo.

CONDIÇÕES DE LEXING E INÍCIO CONDICIONAIS

INSTITUTO FEDERAL
Ceará

- Após declarar um estado, os tokens e as regras são declarados incluindo o nome do estado na declaração de token / regra.

```
t_foo_NUMBER = r'\d+'                         # Token 'NUMBER' in state 'foo'
t_bar_ID     = r'[a-zA-Z_][a-zA-Z0-9_]*'      # Token 'ID' in state 'bar'

def t_foo_newline(t):
    r'\n'
    t.lexer.lineno += 1
```

- Um token pode ser declarado em vários estados, incluindo vários nomes de estados na declaração.

```
t_foo_bar_NUMBER = r'\d+' # Defines token 'NUMBER' in both state 'foo' and 'bar'
```

CONDIÇÕES DE LEXING E INÍCIO CONDICIONAIS

INSTITUTO FEDERAL
Ceará

- Como alternativa, um token pode ser declarado em todos os estados usando '**ANY**' no nome.

```
t_ANY_NUMBER = r'\d+' # Defines a token 'NUMBER' in all states
```

- Se nenhum nome de estado for fornecido, como normalmente acontece, o token será associado a um estado especial '**INITIAL**'.
- Por exemplo, essas duas declarações são idênticas:

```
1  t_NUMBER = r'\d+'
2  t_INITIAL_NUMBER = r'\d+'
```

CONDIÇÕES DE LEXING E INÍCIO CONDICIONAIS

INSTITUTO FEDERAL
Ceará

- Os estados estão associados às declarações especiais **t_ignore, t_error** () e **t_eof** ().
- Se um estado trata isso de maneira diferente, pode-se declarar:

```
1  t_foo_ignore = " \t\n" # Ignored characters for state 'foo'
2
3  def t_bar_error(t):   # Special error handler for state 'bar'
4      pass
```

- Por padrão, o lexing opera no estado '**INITIAL**', que inclui todos os tokens normalmente definidos.
- Isso é transparente para usuários que não estão usando estados diferentes.

CONDIÇÕES DE LEXING E INÍCIO CONDICIONAIS

INSTITUTO FEDERAL
Ceará

- Se, durante a lexing ou análise, precisar alterar o estado da lexing, use o método **begin** ().

```
1 def t_begin_foo(t):
2         r'start_foo'
3         t.lexer.begin('foo') # Starts 'foo' state
```

- Para sair de um estado, use **begin** () para retornar ao estado inicial.

```
1 def t_foo_end(t):
2         r'end_foo'
3         t.lexer.begin('INITIAL')   # Back to the initial state
```

CONDIÇÕES DE LEXING E INÍCIO CONDICIONAIS

INSTITUTO FEDERAL
Ceará

- A gestão de estados pode ser feito com uma pilha. Por exemplo:

```
1  def t_begin_foo(t):
2      r'start_foo'
3      t.lexer.push_state('foo') # Starts 'foo' state
4
5  def t_foo_end(t):
6      r'end_foo'
7      t.lexer.pop_state() # Back to the previous state
```

- Isso é útil em situações em que existem muitas maneiras de inserir um novo estado lexing e deseja voltar ao estado anterior posteriormente.

EXEMPLO DE APLICAÇÃO

INSTITUTO FEDERAL
Ceará

Descrição: Ler seções do código C arbitrário entre chaves.

- Sempre que encontrar uma chave inicial '{', leia todo o código incluído até a chave final '}' e retorná-lo como uma string.
- Fazer isso com uma regra de expressão regular normal é quase impossível.
- As chaves podem ser aninhadas e podem ser incluídas em comentários e cadeias.
- Simplesmente combinar o primeiro caractere '}' correspondente não é bom o suficiente.

EXEMPLO DE APLICAÇÃO

INSTITUTO FEDERAL
Ceará

```
# Declare the state
states = (
   ('ccode','exclusive'),
)

# Match the first {. Enter ccode state.
def t_ccode(t):
    r'\{'
    t.lexer.code_start = t.lexer.lexpos  # Record the starting position
    t.lexer.level = 1                    # Initial brace level
    t.lexer.begin('ccode')               # Enter 'ccode' state

# Rules for the ccode state
def t_ccode_lbrace(t):
    r'\{'
    t.lexer.level +=1
```

EXEMPLO DE APLICAÇÃO

INSTITUTO FEDERAL
Ceará

```
18  def t_ccode_rbrace(t):
19      r'\}'
20      t.lexer.level -=1
21
22      # If closing brace, return the code fragment
23      if t.lexer.level == 0:
24           t.value = t.lexer.lexdata[t.lexer.code_start:t.lexer.lexpos+1]
25           t.type = "CCODE"
26           t.lexer.lineno += t.value.count('\n')
27           t.lexer.begin('INITIAL')
28           return t
29
30  # C or C++ comment (ignore)
31  def t_ccode_comment(t):
32      r'(/\*(.|\n)*?\*/)|(//.*)'
33      pass
34
35  # C string
36  def t_ccode_string(t):
37     r'\"([^\\\n]|(\\.))*?\"'
```

EXEMPLO DE APLICAÇÃO

INSTITUTO FEDERAL
Ceará

```
39  # C character literal
40  def t_ccode_char(t):
41      r'\'([^\\\n]|(\\.))*?\''
42
43  # Any sequence of non-whitespace characters (not braces, strings)
44  def t_ccode_nonspace(t):
45      r'[^\s\{\}\'\"]+'
46
47  # Ignored characters (whitespace)
48  t_ccode_ignore = " \t\n"
49
50  # For bad characters, we just skip over it
51  def t_ccode_error(t):
52      t.lexer.skip(1)
```

EXEMPLO DE APLICAÇÃO

INSTITUTO FEDERAL
Ceará

Ao ocorrer o 1° '{', o lexer registra a posição inicial e insere um novo estado **'CCODE'**.

As regras correspondem a várias partes da entrada: comentários, sequências de caracteres etc.

As regras descartam o token sem retorno de valor.

Se a chave direita de fechamento for encontrada:

- A regra **t_ccode_rbrace** coleta todo o código usando a posição inicial registrada anterior.
- Armazena-o e retorna um token **'CCODE'** contendo todo esse texto.

Ao retornar o token, o estado lexing é restaurado de volta ao seu estado inicial.

QUESTÕES DIVERSAS



INSTITUTO
FEDERAL
Ceará

TRABALHANDO COM DADOS DE STREAMING

INSTITUTO FEDERAL
Ceará

- O lexer exige que a entrada seja fornecida como uma única sequência de entrada.

Como a maioria das máquinas possui memória mais que suficiente, isso raramente é uma preocupação com o desempenho.

No entanto, o lexer atualmente não pode ser usado com dados de streaming, como arquivos abertos ou soquetes.

Essa limitação é principalmente um efeito colateral do uso do re-módulo.

- Pode-se contornar isso implementando uma regra de manipulação de fim de arquivo **def t_eof** () apropriada.

TRABALHANDO COM DADOS DE STREAMING

INSTITUTO FEDERAL
Ceará

- Neste caso de usar **def t_eof ()**, será preciso:

1. Garantir que os dados sejam alimentados para o lexer de forma a não serem divididos no meio de um token.

2. O lexer deve funcionar corretamente com as sequências Unicode e padrão, bem como para o texto de entrada.

TRABALHANDO COM SINALIZADORES

INSTITUTO FEDERAL
Ceará

- Se precisar fornecer sinalizadores opcionais para a função **re.compile** (), use a opção **reflags** para lex. Por exemplo:

```
lex.lex(reflags=re.UNICODE | re.VERBOSE)
```

- **Nota:** por padrão, os reflags estão definidos como **re.VERBOSE**.
- Se fornecer seus próprios sinalizadores, talvez seja necessário incluí-lo no PLY para preservar seu comportamento normal.

REQUISITOS PARA CONSTRUIR LEXER

INSTITUTO FEDERAL
Ceará

- Para criar um lexer escrito à mão com o **yacc.py**, basta estar em conformidade com os seguintes requisitos:

I. Dever existir um método **token()** que retorne o próximo token ou **None** se não houver mais tokens disponíveis;

II. O método **token** () deve retornar um tok de objeto que possua atributos de tipo e valor;

III. Se o rastreamento de número de linha estiver sendo usado, o token também deverá definir um atributo **lineno**.

DÚVIDAS?



INSTITUTO FEDERAL
Ceará

BIBLIOGRAFIA

INSTITUTO
FEDERAL
Ceará

BIBLIOGRAFIA

BEAZLEY, David M. **PLY (Python Lex-Yacc)**. PLY Version: 3.11. Disponível em: < https://www.dabeaz.com/ply/ply.html#ply_nn17 > Data de acesso: 23 de maio de 2025

